Configurable Cache Memory Subsystem

With RTL-to-GDSII Implementation

Project Report

Submitted by

Soumyadeep Dutta

Bachelor of Technology

VLSI Design

July 2026

1. Abstract

The increasing disparity between processor speed and main memory latency has made cache memories indispensable in modern computing systems. This project presents the design and implementation of a configurable cache memory subsystem using Verilog HDL and demonstrates a complete RTL-to-GDSII physical design flow using open-source EDA tools.

The proposed cache subsystem supports direct-mapped and set-associative organizations, write-back policy, dirty-bit management, LRU replacement, and performance monitoring counters. Functional verification was performed using ModelSim, synthesis was performed using Yosys, and physical design was implemented using OpenROAD with the Nangate45 standard-cell library.

The final implementation achieved timing closure with zero negative slack while producing a manufacturable GDSII layout.

2. Introduction

Modern processors execute billions of instructions per second. However, the access time of off-chip DRAM has not improved at the same rate as processor frequencies. This difference, commonly referred to as the **memory wall**, significantly degrades system performance.

Cache memories are introduced to bridge this gap by storing frequently accessed data closer to the processor. Since cache access times are much smaller than DRAM access times, overall system performance can be significantly improved.

The objective of this project is to design a configurable cache subsystem and demonstrate its complete implementation from RTL specification to physical layout generation.

3. Project Objectives

- Design a configurable cache memory subsystem.
- Implement write-back and dirty-bit handling.
- Implement LRU replacement policy.
- Design performance monitoring counters.
- Demonstrate complete RTL-to-GDSII implementation.
- Achieve timing closure.

4. Cache Memory Fundamentals

4.1. Memory Hierarchy

The memory hierarchy consists of multiple storage levels with different capacities and access times.

- Registers
- L1 Cache
- L2 Cache
- L3 Cache
- Main Memory
- Secondary Storage

4.2. Cache Parameters

The cache architecture is defined by several important parameters.

- Cache Size
- Block Size
- Associativity
- Tag Bits
- Index Bits
- Offset Bits

4.3. Address Breakdown

A 32-bit memory address can be divided into:

```
| Tag | Index | Offset |
```

The relationship between cache parameters is given by:

```
Cache Size =
Number of Sets × Associativity × Block Size
```

5. Proposed Architecture

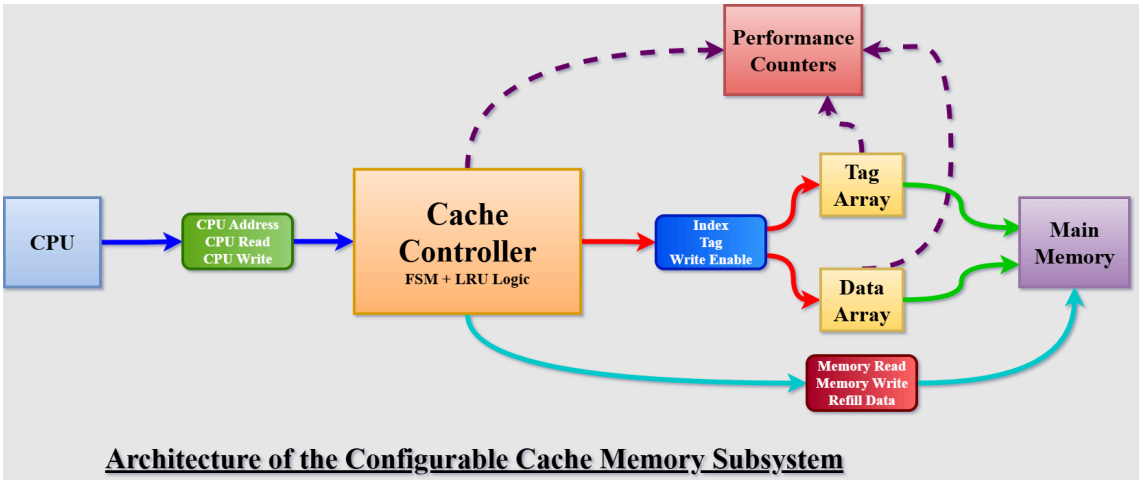

Figure 1: Architecture of the Configurable Cache Memory Subsystem

The proposed architecture consists of the following major modules.

5.1. CPU Interface

The processor communicates with the cache through the following signals:

- clk
- rst
- cpu_read
- cpu_write
- cpu_addr
- cpu_wdata
- cpu_rdata

5.2. Cache Controller

The cache controller performs:

- Tag comparison
- Hit and miss detection
- Refill operations
- Write-back handling
- Dirty eviction
- Replacement decisions

5.3. Tag Array

Stores:

- Tag bits
- Valid bits
- Dirty bits

5.4. Data Array

Stores the cache lines and supports:

- Read operation
- Write operation
- Refill operation

5.5. Performance Counters

Records:

- Read hits
- Read misses
- Write hits
- Write misses

6. RTL Design

The cache subsystem was implemented using Verilog HDL.

The following modules were designed.

- cache_top.v
- cache_controller.v
- tag_array.v
- data_array.v
- main_memory.v
- lru_logic.v
- performance_counters.v

6.1. Cache Controller FSM

The controller operates using the following sequence:

```
IDLE
↓
TAG_CHECK
↓
READ_MISS
↓
REFILL
↓
WRITE_BACK
```

The controller coordinates memory transactions and updates cache metadata.

7. Functional Verification

The design was functionally verified using ModelSim.

7.1. Cache Read Miss and Refill

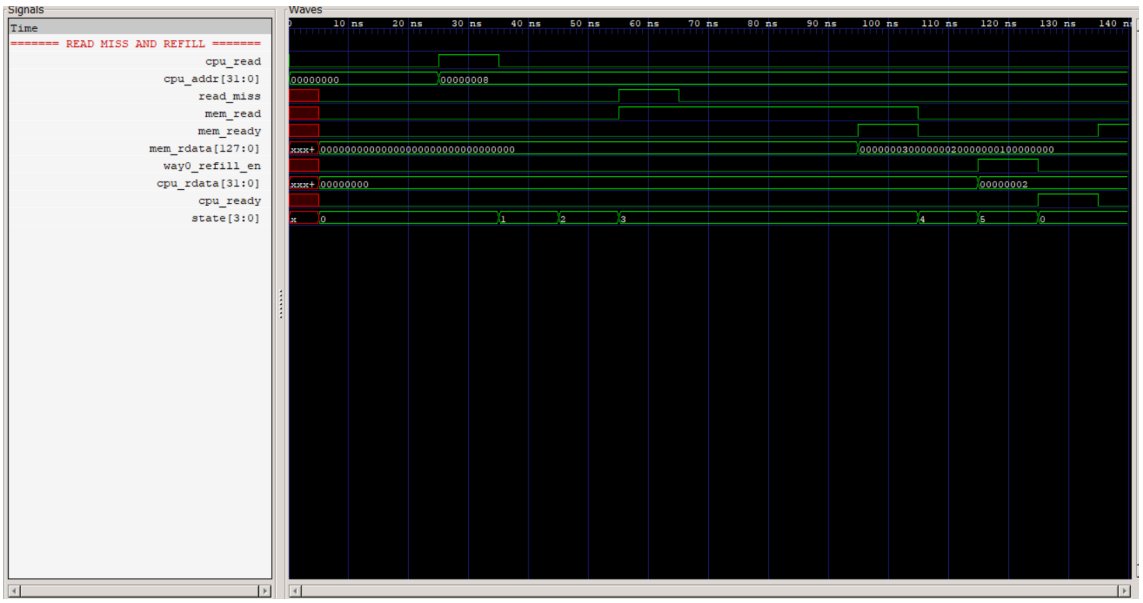

Figure 2: Cache Read Miss and Refill

The first memory access results in a cache miss and initiates a memory refill operation.

7.2. Cache Read Hit

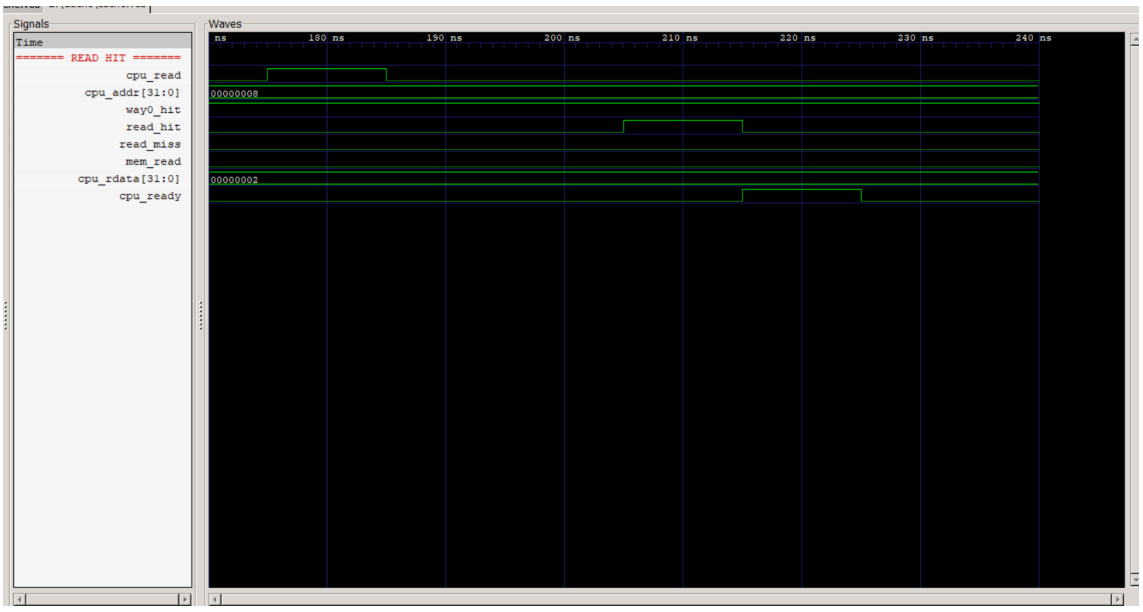

Figure 3: Cache Read Hit

Subsequent accesses to the same address produce cache hits.

7.3. Write Hit and Dirty Bit Update

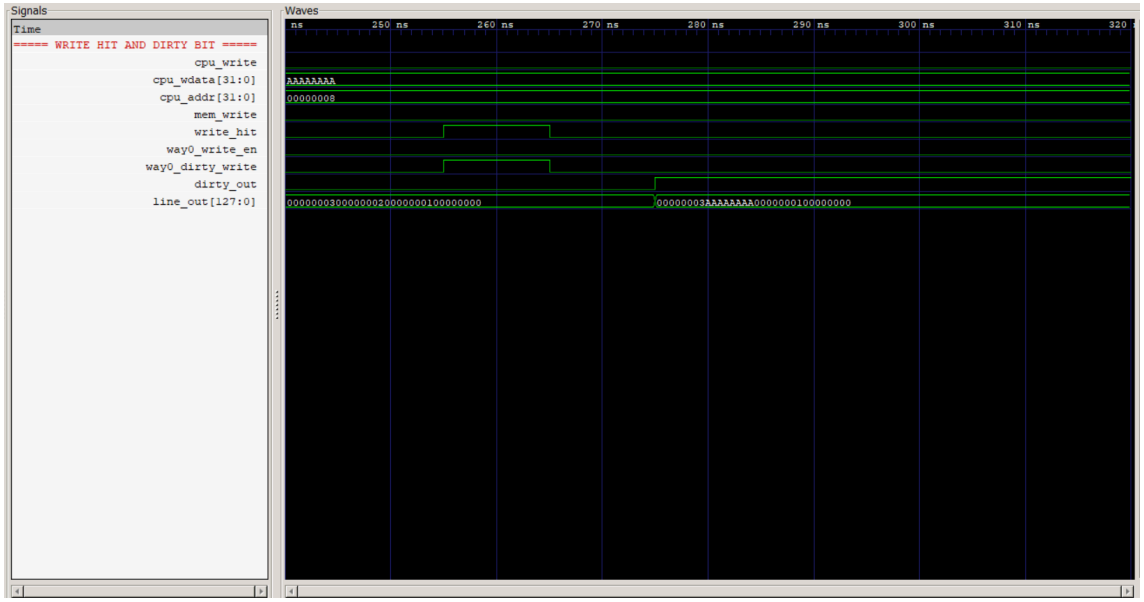

Figure 4: Write Hit and Dirty Bit Update

The cache line is updated and marked dirty.

7.4. LRU-Based Cache Replacement

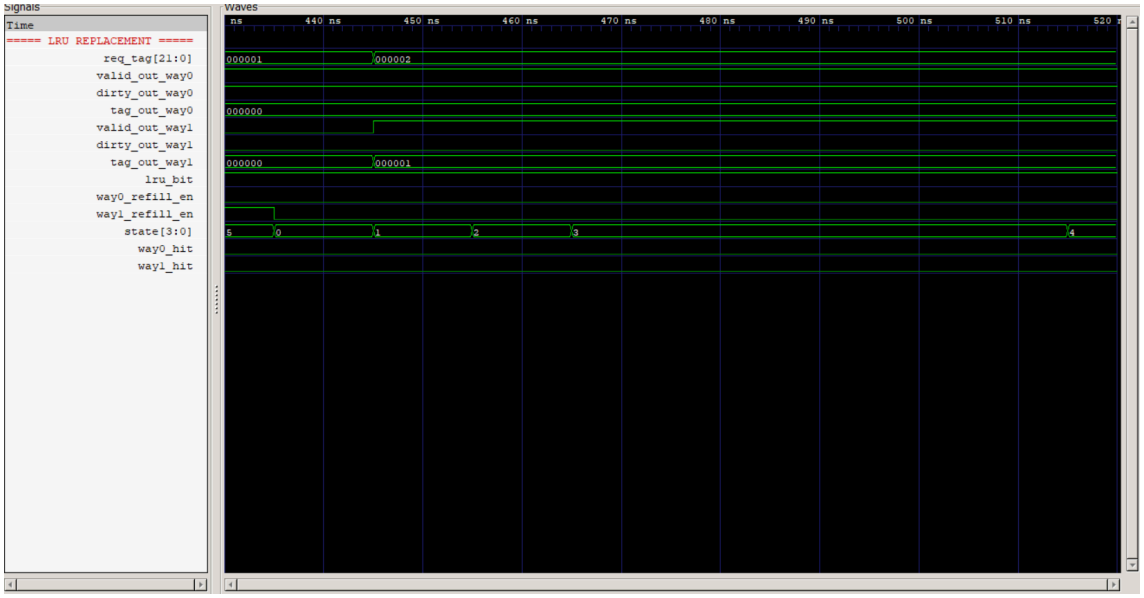

Figure 5: LRU-Based Cache Replacement

The least recently used cache line is selected as the victim during replacement.

7.5. Performance Counters

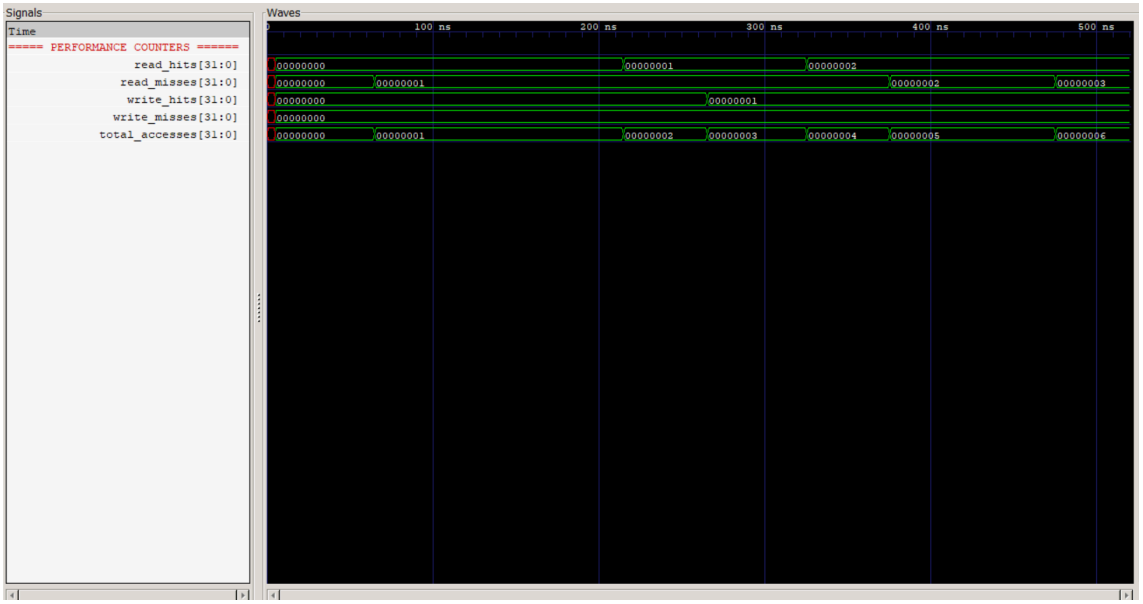

Figure 6: Performance Counters

The counters record cache statistics during operation.

8. Logic Synthesis

The RTL design was synthesized using the Yosys Open Synthesis Suite.

```
18. Executing Verilog backend.

18.1. Executing BMUXMAP pass.

18.2. Executing DEMUXMAP pass.
Dumping module '\cache_controller'.
Dumping module '\cache_top'.
Dumping module '\data_array'.
Dumping module '\lru_logic'.
Dumping module '\main_memory'.
Dumping module '\performance_counters'.
Dumping module '\tag_array'.

End of script. Logfile hash: 7a3ccbbfcf, CPU: user 19.21s system 0.47s, MEM: 190.24 MB peak
Yosys 0.52 (git sha1 fee39a3284c90249e1d9684cf6944ffbbcbb8f90)
Time spent: 29% 13x opt_dff (5 sec), 24% 19x opt_expr (4 sec), 20% 16x opt_clean (3 sec), ...
soumyadeep02@LAPTOP-5UM5809C:~/cache_asic$
```

Figure 7: Yosys Synthesis Results

```
=== design hierarchy ===

   cache_top                       1
     cache_controller              1
     data_array                    2
     lru_logic                     1
     main_memory                   1
     performance_counters          1
     tag_array                     2

   Number of wires:              13510
   Number of wire bits:          83207
   Number of public wires:         597
   Number of public wire bits:   13928
   Number of ports:                134
   Number of port bits:           2382
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:              76984
     $_AND_                       5445
     $_DFFE_PP_                  10399
     $_MUX_                      55467
     $_NOT_                        356
     $_OR_                        4044
     $_SDFFE_PP0N_                 279
     $_SDFFE_PP0P_                 433
     $_SDFF_PP0_                   322
     $_XOR_                        239
```

Figure 8: Yosys Statistics

The synthesis process converts RTL descriptions into gate-level netlists and performs logic optimization.

9. FPGA Compilation Results

Flow Summary

<<Filter>>

Flow Status	Successful - Fri Jun 19 16:49:07 2026
Quartus Prime Version	20.1.1 Build 720 11/11/2020 SJ Lite Edition
Revision Name	cache_top
Top-level Entity Name	cache_top
Family	MAX 10
Device	10M50DCF672I7G
Timing Models	Final
Total logic elements	19,715 / 49,760 (40 %)
Total registers	11413
Total pins	265 / 500 (53 %)
Total virtual pins	0
Total memory bits	0 / 1,677,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 288 (0 %)
Total PLLs	0 / 4 (0 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0

Figure 9: Quartus Compilation Successful

Fitter Resource Usage Summary

<<Filter>>

	Resource	Usage
1	Total logic elements	19,715 / 4...0 (40 %)
2		
3	Logic element usage by number of LUT inputs	
1	-- 4 input functions	12449
2	-- 3 input functions	982
3	-- <=2 input functions	536
4	-- Register only	5748
4		
5	Logic elements by mode	
1	-- normal mode	13812
2	-- arithmetic mode	155
6		
7	Total registers*	11,413 / 5...9 (22 %)
1	-- Dedicated logic registers	11,413 / 4...0 (23 %)
2	-- I/O registers	0 / 2,449 (0 %)
8		
9	Total LABs: partially or completely used	1,776 / 3,110 (57 %)
10	Virtual pins	0
11	I/O pins	265 / 500 (53 %)
12		
13	M9Ks	0 / 182 (0 %)

* Register count does not include registers inside RAM blocks or DSP blocks.

Figure 10: FPGA Resource Utilization

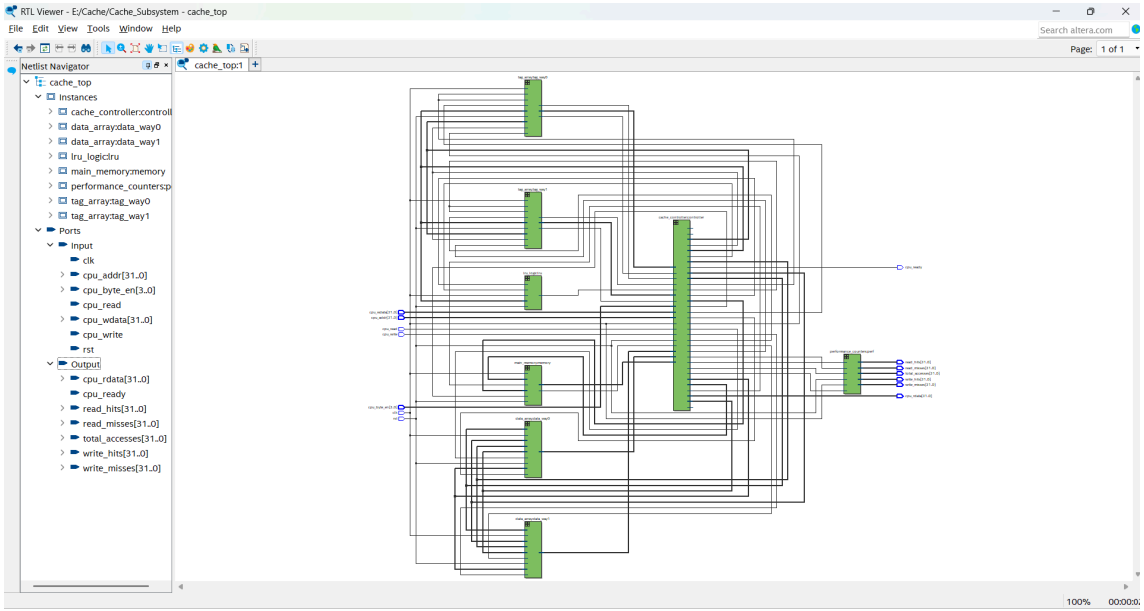

Figure 11: RTL Viewer

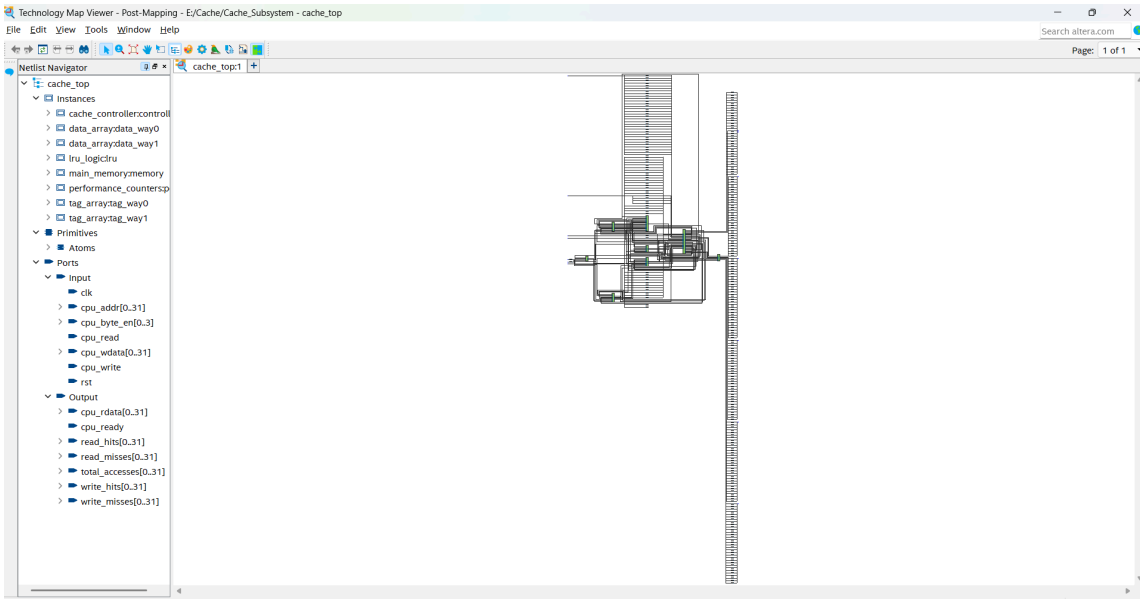

Figure 12: Technology Mapping

10. RTL-to-GDSII Physical Design Flow

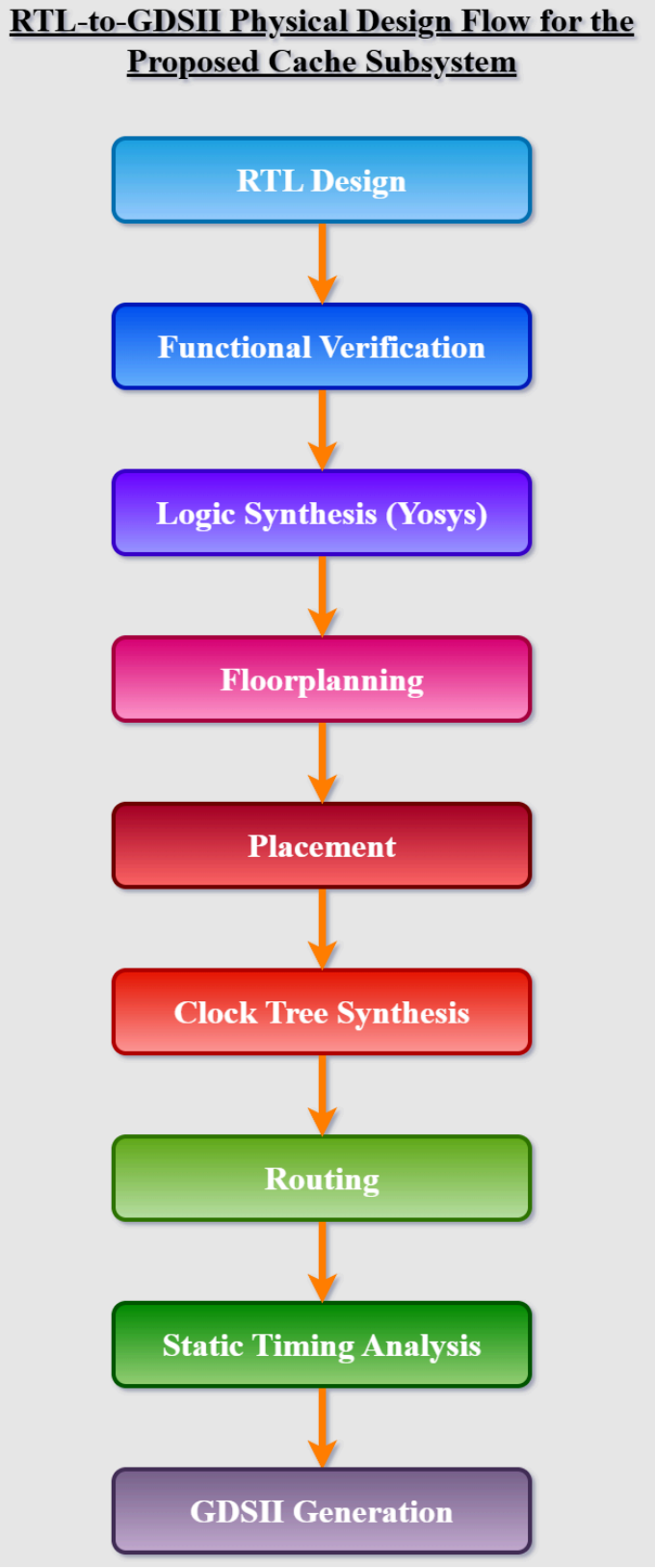

Figure 13: RTL-to-GDSII ASIC Design Flow

The complete physical design flow consists of:

1. Logic Synthesis

2. Floorplanning
3. Placement
4. Clock Tree Synthesis
5. Routing
6. Static Timing Analysis
7. GDSII Generation

11. Physical Design Implementation

11.1. Floorplan

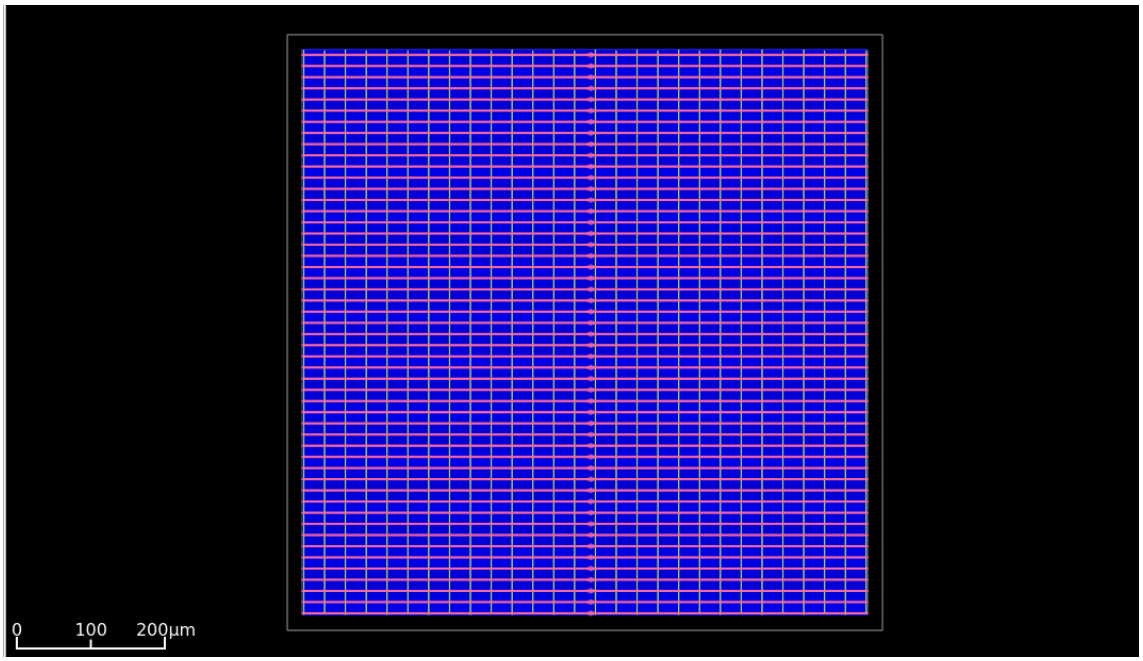

Figure 14: Floorplan and Power Distribution Network

11.2. Placement

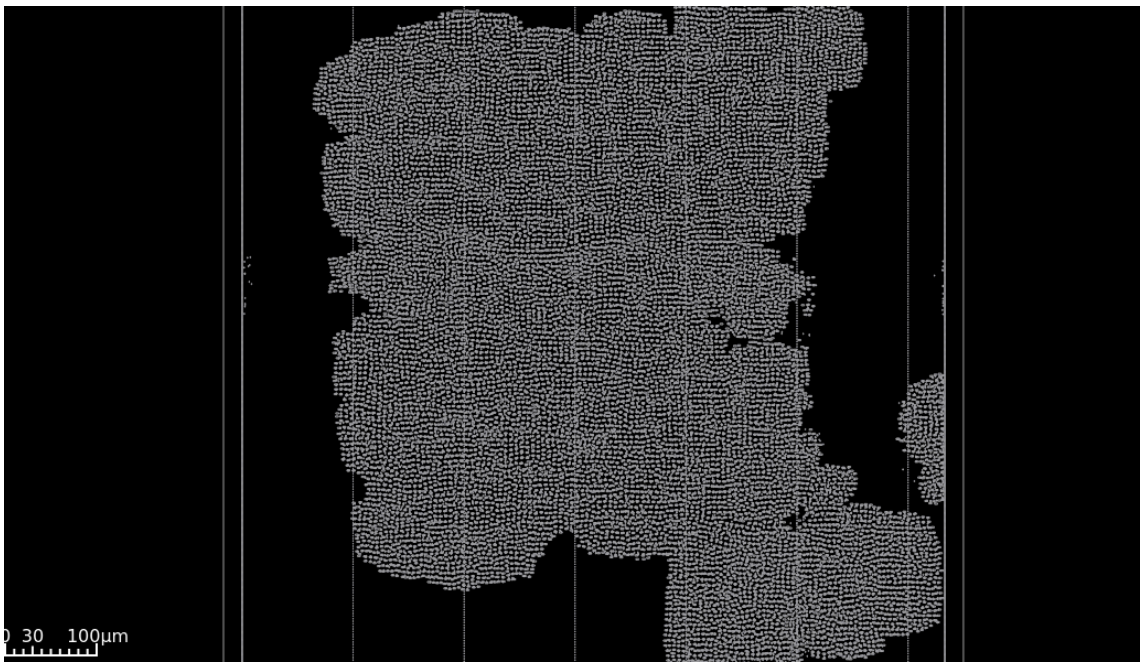

Figure 15: Standard Cell Placement

11.3. Routing

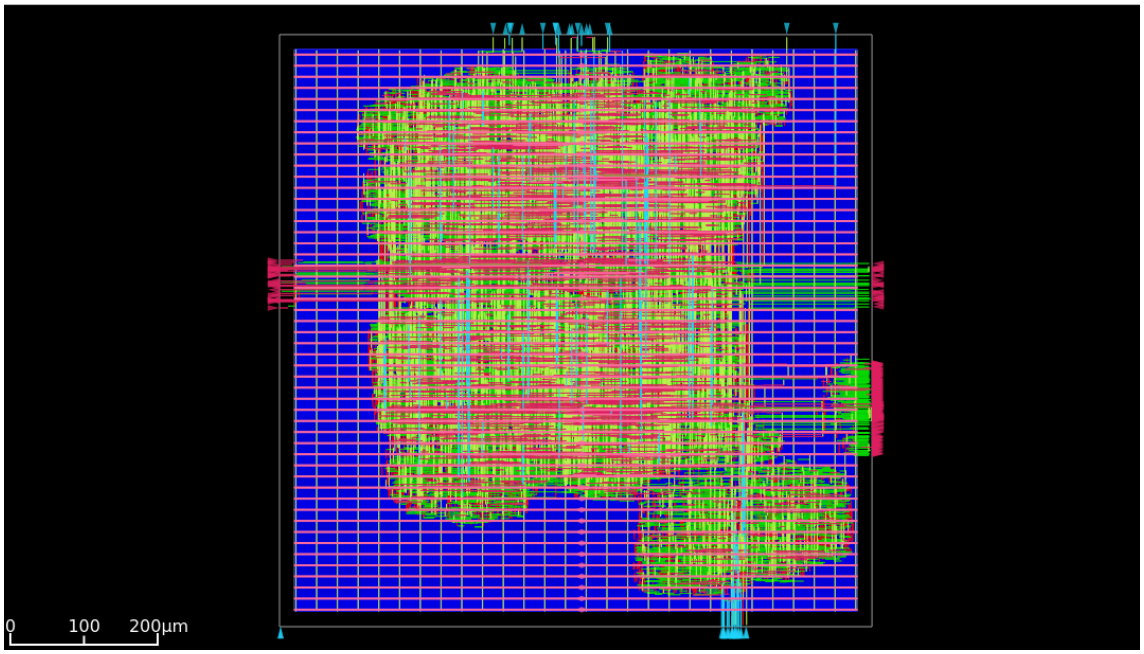

Figure 16: Final Routed Layout

12. Physical Design Results

The final physical implementation produced the following results:

Parameter	Result
Technology Node	Nangate45 (45 nm)
Final Design Area	101,684 μm²
Core Utilization	18%
Standard Cell Count	189,626
WNS (Worst Negative Slack)	0.00 ns
TNS (Total Negative Slack)	0.00 ns
Timing Closure	Achieved
Physical Design Flow	RTL -> Synthesis -> Floorplanning -> Placement -> CTS -> Routing -> GDSII

Figure 17: Physical Design Summary

The design successfully achieved timing closure without setup violations.

13. Challenges Faced

Several practical challenges were encountered:

- OpenROAD dependency installation issues.
- OR-Tools and Lemon package configuration errors.
- ODB database version mismatch.
- Floorplan site initialization problems.
- Integration of the complete RTL-to-GDSII flow.

These challenges provided valuable experience with industrial EDA workflows.

14. Conclusion and Future Work

A configurable cache memory subsystem was successfully designed, verified, synthesized, and physically implemented using a complete open-source ASIC flow.

Future improvements include:

- Four-way set associative cache.
- Multi-level cache hierarchy.
- Hardware prefetching.
- ECC support.
- Integration with a RISC-V processor.

15. References

[1] OpenROAD Project Documentation.

[2] Yosys Open Synthesis Suite Documentation.

[3] Nangate45 Open Cell Library.

[4] David Harris and Sarah Harris, **Digital Design and Computer Architecture**.

[5] Patterson and Hennessy, **Computer Organization and Design**.